PROGRAMACIÓN AVANZADA

Hibernate

Parte I



Ing. Matias Macrino

HIBERNATE

Conceptos Básicos:

✔***ORM:*** de sus siglas del ingles "***Object Relational Mapping"*** y en su traducción al castellano ***"Mapeo de Objetos Relacional***". Es el código que escribimos para guardar el valor de nuestro ***Objeto*** que se instancia desde una clase en un base de datos relacional.

✔En pocas palabras es un Framework de persistencia de nuestros datos en una base de datos relacional.

HIBERNATE

Conceptos Básicos (ORM vs JDBC):

✔Todo lo que permite realizar el ***ORM***, es lo mismo que puedo hacer desde JDBC, entonces ¿Qué ventaja tengo en usar esta tecnología?.

✔No es necesario estar usando el método ***setString(int parameterIndex, String value)***, para poder pasarle los datos a un ***PreparedStatement*** y por fin poder almacenarlos en la base de datos.

✔No va hacer necesario el control de datos que no sean nulos.

✔Cuando se modifique alguna tabla en la que estas trabajando, no vas a tener que estar revisando clase por clase en donde interviene tu objeto para poder volver a persistir correctamente los datos.

HIBERNATE

Conceptos Básicos:

✔***Hibernate:*** es un ***Framework*** de persistencia, el cual se encuentra basado en estándares de ***JPA (Java Persistence Api).***

✔Para la persistencia de objetos Java en la base de datos relacional, hay que indicar como se realizara. Con lo cual nos encontramos con dos métodos:

- ***Archivos XML***
- ***Anotaciones en el código.***

HIBERNATE

Conceptos Básicos:

✔ Archivos XML: Los archivos XML que usaremos son propios de ***hibernate***, por lo cual al verlos conoceremos como funciona realmente ***hibernate.***

✔ Anotaciones en el código: Las anotaciones que utiliza ***hibernate*** son actualmente las del estándar de ***JPA***. Por lo cual al ver la anotaciones de ***hibernate*** estaremos conociendo las propias de ***JPA.***

HIBERNATE

Instalación de Hibernate:

✔***documentation:*** toda la documentación correspondiente a ***Hibernate***

✔***project:*** contiene principalmente el código fuente y archivos de configuración de las distintas base de datos.

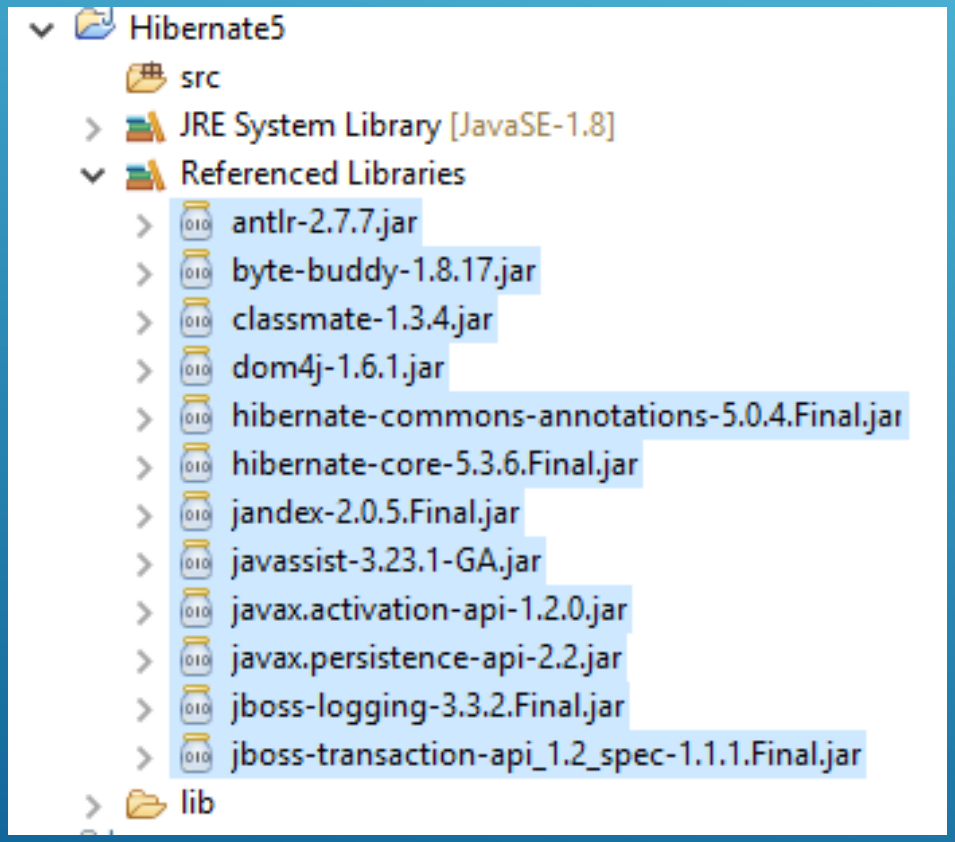
✔***lib:*** contiene las librerías (archivos *.jar) java con el código de ***Hibernate*** compilado.

- ***lib\required:*** contiene los jars correspondientes del ***Hibernate-core*** que siempre deben usarse. Es decir siempre debemos incluir estos jars en todos nuestros proyectos.
- ***lib\optional:*** contiene las librerías que añaden nuevas funcionalidades a ***hibernate***.
- ***lib\envers:*** contiene librerías que permiten realizar auditorías sobre los datos que se persisten.
- ***lib\spatial – lib\osgi:*** son librerías que aparecen a partir de su versión 5 y añaden nuevas funcionalidades a ***Hibernate***.
- ***lib\jpa-metamodel-generator:*** contiene las librerías necesarias para generar el modelo de seguridad de JPA.

HIBERNATE

Instalación de Hibernate:

✔Crear proyecto java, en el cual deberemos de incluir las librerías que se encuentran dentro de las carpetas lib\required



HIBERNATE

Instalación de Hibernate (configuración):

- ✔ Como en cualquier otro Framework es necesario su configuración, para su correcto funcionamiento.

- ✔ ¿Qué es necesario configurar?
  - ✔ Datos de conexión a la base de datos.
  - ✔ Ubicación de las clases a persistir.
  - ✔ Ubicación de los archivos XML (hbm.xml).
  - ✔ Pool de conexiones a usar.
  - ✔ Nivel de log.

HIBERNATE

Instalación de Hibernate (configuración):

- La configuración de ***hibernate*** es usando un archivo XML de configuración, el cual debe de se llamado "***hibernate.cfg.xml***" y guardado en el paquete raíz de las clases Java del proyecto, es decir fuera de cualquier paquete creado. Carpeta raíz del proyecto "***src***".

- Dentro del mismo se encontrara la siguiente información:
  - Propiedades de configuración.
  - Las clases que se quieren mapear.

HIBERNATE

Instalación de Hibernate (hibernate.cfg.xml):

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration SYSTEM "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.microsoft.sqlserver.jdbc.SQLServerDriver</property>
        <property name="hibernate.connection.url">jdbc:sqlserver://127.0.0.1\MSSQLSERVER:1433;DatabaseName=Hibernate</property>
        <property name="hibernate.connection.username">sa</property>
        <property name="hibernate.connection.password">123456</property>
        <property name="hibernate.dialect">org.hibernate.dialect.SQLServerDialect</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.current_session_context_class">thread</property>
        <property name="show_sql">true</property>

        <mapping class="negocio.Profesor"/>
        <mapping resource="ejemplo01/Profesor.hbm.xml"/>

    </session-factory>
</hibernate-configuration>
```

HIBERNATE

Instalación de Hibernate (configuración):

- ✔ En la línea 1 se encuentra la definición de que es un archivo XML.
- ✔ En la línea 4 se encuentra el tag raíz del XML "***<hibernate-configuration>***"
- ✔ En la línea 5 se encuentra el tag que contendrá toda la configuración de hibernate "***<session-factory>***"
- ✔ De las líneas 6 a la 13 se encuentran todas las propiedades de configuración mediante tag "***<property>***".
  - El atributo "***name***" contiene el nombre de la propiedad de configuración.
  - El contenido del tag define el valor que tendrá dicha propiedad.
- ✔ De las líneas 15 a la 16 se encuentra el tag "***<mapping>***", el cual se utiliza para indicarle a hibernate las clases a utilizar para la persistencia.
  - El atributo "resources" contiene la ubicación del archivo XML.
  - El atributo "class" contiene la ubicación de la clase, con sus anotaciones.

HIBERNATE

Instalación de Hibernate (property - mapping):

✔***hibernate.connection.driver_class***: Contiene el nombre de clase de la base de datos a utilizar. Es lo mismo que se utiliza en ***Class.forName*** en ***JDBC***

✔***hibernate.connection.url***: La url de conexión a la base de datos, tal cual como se hace para ***JDBC***

✔***hibernate.connection.username***: Usuario de la BD.

✔***hibernate.connection.password***: Password de la BD.

✔***hibernate.dialect***: El lenguaje de SQL que utilizara hibernate contra la BD. Es opcional ya que hibernate lo puede deducir a partir de los datos de la conexión.

HIBERNATE

Instalación de Hibernate (property - mapping):

✔***hibernate.hbm2ddl.auto***: Valida o genera automáticamente el esquema DDL al motor de BD. Sus posibles valores son: ***validate*** | ***update*** | ***create*** | ***create-drop.***
Hibernate, no crea la Base de Datos, solamente las tablas.

✔***show_sql***: Propiedad opcional, indica si se mostrara por consola la sentencia SQL que lanza hibernate contra el motor de BD. Sus posibles valores son true o false

✔<mapping>:

- ***resource***: Se utiliza para cuando se creo el archivo XML de mapeo "hbm.xml" para poder persistir la clase java a la base de datos.
- ***class***: Se utiliza para cuando se creo una clase java con anotaciones JPA en vez de utilizar el archivo XML.

HIBERNATE

Mapeo de entidades:

✔Una entidad la llamaremos a una simple clase Java que será la que termine persistiendo contra el motor de Base de Datos.

✔La clase Java deben de contener las siguientes características:

- Tener si o si el constructor por defecto.
- Por cada una de las variables de instancias de la clase, su correspondiente método get y set.
- Implementar la interfaz ***Serializable***.

✔Esta clase Java no puede ser persistida todavía, ya que queda definir toda la metainformacion relativa a la clase, como por ejemplo como se mapea la clase en una BD Relacional indicando su tabla, clave primaria, columnas, etc.

HIBERNATE

Mapeo de entidades (archivo XML):

- ✔Para cada una de las clases Java que vayamos a persistir se tiene que generar un archvio XML, con la información que permitirá mapear la clase a una BD Relacional.
- ✔Este archivo se debe de encontrar en el mismo paquete en donde se encontrara nuestra clase Java.
- ✔El archivo llevara el nombre de nuestra clase Java mas "hbm.xml", ejemplo Alumno.hbm.xml

```
1   <?xml version="1.0"?>
2   <!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
3   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
4
5   <hibernate-mapping>
6       <class name="edu.usal.negocio.dto.Alumno" table="Alumnos">
7           <id column = "codigo_alumno" name="cod_Alumno" type="integer"  >
8               <generator class="identity"/>
9           </id>
10          <property name="nombre" column="nombres" />
11          <property name="apellido" column="apellido"/>
12      </class>
13  </hibernate-mapping>
```

HIBERNATE

Mapeo de entidades (anotaciones):

✔Desde ya hace un tiempo en Java se empezó a utilizar el termino de anotaciones dentro del propio código. Lo cual nos evita el escribir archivos XML de configuración, siendo los mismos demasiados largos de escribir, repetitivos y por cada Framework a utilizar un XML distinto.

✔Estas anotaciones nos permite especificar de una forma mas compacta y sencilla la información de mapeo de una clase Java.

✔Las anotaciones a utilizar son las del estándar ***JPA*** que se encuentran dentro del paquete ***javax.persistence***

HIBERNATE

Mapeo de entidades (anotaciones):

```java
@Entity
@Table(name="Alumnos")
public class Alumnos implements Serializable {

    @Id
    @Column(name = "codigo_alumno")
    private int cod_alumno
    @Column(name = "nombres")
    private String nombre;
    @Column(name = "apellido")
    private String apellido;

    public Alumnos () { }

    public void setCod_alumno(int cod_alumno) {
        this.cod_alumno = cod_alumno;
    }
    public int getCod_alumno() {
        return this.cod_alumno;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getNombre() {
        return this.nombre;
    }
    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
    public String getApellido() {
        return this.apellido;
    }
}
```

@Entity: Anotación estándar de JPA. Se aplica a la clase e indica que dicha clase es una entidad a persistir.

@Table: Anotación estándar JPA. Se aplica a la clase e indica el nombre de la tabla de la BD donde persiste la clase.

@Id: Anotación estándar JPA. Se aplica a una variable de instancia e indica que este atributo el Clave Primaria.

@Column: Anotación estándar JPA. Se aplica a una variable de instancia e indica el nombre de la columna que se persistirá.

HIBERNATE

Usando Hibernate:

✔Para utilizar código Java y poder persistir una clase/entidad, las clases que mas se usaran de ***Hibernate*** es ***Sesssion*** (***org.hibernate.session***). La cual contiene métodos para leer, guardar, modificar o borrar entidades sobre la base de datos.

✔Antes de poder utilizar sus métodos, hace falta crear un objeto ***SessionFactory*** (***org.hibernate.SessionFactory***) que mediante su método ***SessionFactory.openSession();*** nos dará acceso a ***Session***.

HIBERNATE

Usando Hibernate (SessionFactory):

✔La forma de crear un objeto ***SessionFactory*** es mediante otro objeto ***MetdaDataSources*** (***org.hibernate.boot.MetadataSources***)

✔***Objeto MetdaDataSources:*** se encargara de crear un objeto ***SessionFactory*** mediante la información proveniente del objeto ***StandardServiceRegistry*** (***org.hibernate.boot.registry.StandardServiceRegistry***)

✔***Objeto StandardServiceRegistry:*** se encargara de leer el archivo XML de configuración de ***Hibernate*** (***hibernate.cfg.xml***) que se encuentra en el directorio raíz de nuestro proyecto (***src***)

HIBERNATE

Usando Hibernate (SessionFactory):

```
1   SessionFactory sessionFactory;
2   StandardServiceRegistry registry = new StandardServiceRegistryBuilder().configure().build();
3   sessionFactory = new MetadataSources(registry).buildMetadata().buildSessionFactory();
4
```

- En la línea 2 vemos como se realiza la lectura del archivo de configuración.
- En la línea 3, mediante el método a través del objeto ***MetadataSources*** se ve como se crea el objeto ***SessionFactory*** en función de la configuración obtenida del objeto ***StandarServiceRegistry***

✔Todo el código visto en este ejemplo deberá de realizarse una única vez en la aplicación y guardar la referencia en ***SessionFactory***, para su posterior uso durante toda la aplicación

✔Al finalizar la aplicación debe de llamarse al método ***close()*** (***SessionFactory.close();***)

HIBERNATE

Usando Hibernate (Session):

✔Una vez creado el objeto ***SessionFactory*** ya podemos obtener la sesión para instanciar el objeto ***Session*** para trabajar con Hibernate.

✔Para crear un objeto ***Session*** es tan sencillo que utilizar el método ***openSession();***

- Ejemplo: ***Session session = sessionFactory.openSession();***

✔Una vez que obtuvimos la sesión ya podemos trabajar con Hibernate, así persistiendo la clase y una vez finalizada se deberá de cerrar con el método ***close():***

- ✔Ejemplo: ***session.close();***

HIBERNATE

Usando Hibernate (Transacciones):

✔Al igual que se hace con JDBC o con cualquier tecnología que tenga un impacto sobre un Base de Datos, se utiliza una ***transacción (acción atómica).***

✔Para trabajar con ***Hibernate*** con una Base de Datos ***usamos las transacciones.*** En ***Hibernate*** es tan sencillo como:

HIBERNATE

Usando Hibernate (Transacciones):

✔Crear una nueva transacción, utilizando el método ***beginTransaction()*** de la sesión creada.

- ✔ Ejemplo: ***session.beginTransaction();***

✔Una vez ocurrida la acción atómica realizamos un commit de la transacción actual llamando al método ***commit().***

- ✔ Ejemplo: ***session.getTransaction().commit();***

✔Si no ocurre correctamente la acción atómica se realiza un rollback de la transacción actual llamando al método ***rollback();***

- ✔ Ejemplo: ***session.getTransaction().rollback();***

HIBERNATE

Usando Hibernate (CRUD):

✔Ya contamos con todo lo necesario para poder trabajar con Hibernate y realizar las operaciones fundamentales en una Base de Datos CRUD:

- ✔ ***C***reate: guardar un nuevo objeto en la Base de Datos.
- ✔ ***R***ead: Leer los datos de un objeto en la Base de Datos.
- ✔ ***U***pdate: Actualizar los datos de un objeto en la Base de Datos.
- ✔ ***D***elete: Borrar los datos de un objeto en la Base de Datos.

✔Estas operaciones son tan sencillas de implementar con tan solo usar desde Hibernate un único método para cada una de ellas.

HIBERNATE

Usando Hibernate (CRUD):

✔ ***Create – Guardar:*** Para este operación usaremos el método ***save(Object object)*** de la sesión pasándole como único argumento un objeto a persistir en la base de datos.

- ✔ Ejemplo:

```
1   Alumnos alumno = new Alumnos(1, "Nombre Alumno", "Apellido");
2
3   Session session = sessionFactory.openSession();
4   session.beginTransaction();
5
6   session.save(alumno);
7
8   session.getTransaction().commit();
9   session.close();
10
```

✔ En la línea 6 se realiza la persistencia del objeto en la Base de Datos.

HIBERNATE

Usando Hibernate (CRUD):

✔***Read - Leer:*** Para este operación usaremos el método ***get(Class, Serializable)***, al que le debemos de pasar la clase queremos leer y su clave primaria.

- ✔ Ejemplo:

```
Alumnos alumno = (Alumnos)session.get(alumno.class, 1);
```

✔Dicho método solo permite leer un único objeto de la Base de Datos a partir de su clave primaria.

✔Dos características a tener en cuenta:

- ***Uso del casteo:*** Es necesario ya que el método ***get()*** se usa para cualquier tipo de entidad, así no sabiendo Java que tipo de dato retornar.
- *Uso de la propiedad .class:* Forma que definió Java para pasar un objeto de la clase ***Java.lang.Class***

HIBERNATE

Usando Hibernate (CRUD):

✔***Update - Actualizar:*** Para este operación usaremos el método ***update(Object object)*** de la sesión pasándole como único argumento un objeto a actualizar en la base de datos.

- ✔ Ejemplo:

```
1   Alumnos alumno = new Alumnos(1, "Nombres del Alumno", "Apellido");
2
3   Session session = sessionFactory.openSession();
4   session.beginTransaction();
5
6   session.update(alumno);
7
8   session.getTransaction().commit();
9   session.close();
10
```

✔En la línea 6 se realiza la actualización del objeto en la Base de Datos.

HIBERNATE

Usando Hibernate (CRUD):

✔***Delete - Borrar:*** Para este operación usaremos el método ***delete(Object object)*** de la sesión pasándole como único argumento un objeto a actualizar en la base de datos.

- ✔ Ejemplo:

```
1   Alumnos alumno = new Alumnos(1, null, null);
2
3   Session session = sessionFactory.openSession();
4   session.beginTransaction();
5
6   session.delete(alumno);
7
8   session.getTransaction().commit();
9   session.close();
```

✔En la línea 6 se realiza borrado del objeto en la Base de Datos.

HIBERNATE

Usando Hibernate (CRUD):

✔ ***Alternativa a Create - Update:*** Muchas veces resulta cómodo al desarrollar una aplicación no tener que estar pendiente de si el objeto va a guardar o actualizarse. Para esto ***Hibernate*** dispone del método ***saveOrUpdate(Object object)***

✔ Ejemplo:

```
1   Alumnos alumno = new Alumnos(1, "Nombres del Alumno", "Apellido");
2
3   Session session = sessionFactory.openSession();
4   session.beginTransaction();
5
6   session.saveOrUpdate(alumno);
7
8   session.getTransaction().commit();
9   session.close();
```

✔En la línea 6 lo que se realiza en este caso depende de si existe o no el registro en la base de datos el objeto con Id = 1. Se realiza el INSERT o UPDATE.

HIBERNATE

Usando Hibernate (CRUD):

✔***Como se puede observar en todos los ejemplos, con tan solo una línea de código podemos hacer un INSERT, DELETE o UPDATE sobre la Base de Datos.***

✔***Con JDBC debemos realizar muchos pasos previos y utilizar mas líneas de código para poder persistir un clase Java en la Base de Datos.***

HIBERNATE

Usando Hibernate con JPA (Java Persistence API):

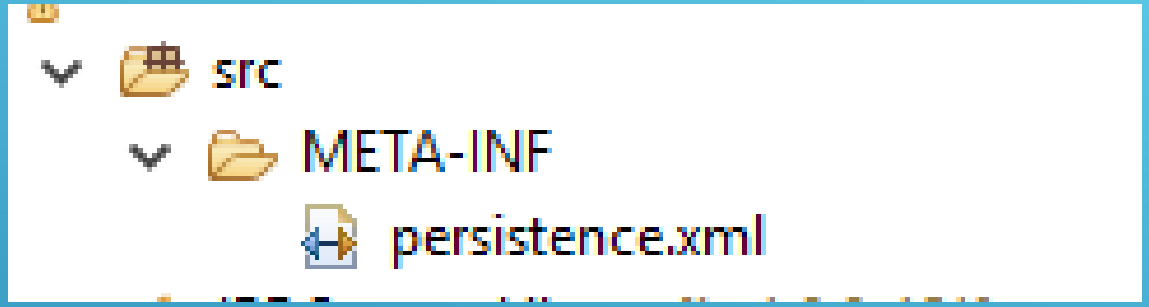
✔Además de usar la configuración propia de Hibernate, se puede utilizar las configuraciones propias definidas por el estándar de JPA.

✔Hibernate para su configuración utiliza el archivo ***hibernate.cfg.xml.***

✔***JPA*** define que su archivo de configuración es ***persistence.xml*** el cual se debe de ubicar dentro de la siguiente ruta ***META-INF\persistence.xml***

HIBERNATE

Usando Hibernate con JPA (persistence.xml):




```xml
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://java.sun.com/xml/ns/persistence http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd"
        version="2.0">
    <persistence-unit name="edu.usal.jpa">
        <class>edu.usal.negocio.dto.Alumno</class>
        <properties>
            <property name="javax.persistence.jdbc.driver" value="com.microsoft.sqlserver.jdbc.SQLServerDriver"/>
            <property name="javax.persistence.jdbc.url" value="jdbc:sqlserver://127.0.0.1:1433;DatabaseName=Hibernate2"/>
            <property name="javax.persistence.jdbc.user" value="sa"/>
            <property name="javax.persistence.jdbc.password" value="123456"/>
            <property name="javax.persistence.schema-generation.database.action" value="update"/>
            <property name = "hibernate.show_sql" value = "true" />
        </properties>
    </persistence-unit>
</persistence>
```

HIBERNATE

Usando Hibernate con JPA (persistence.xml):

- ✔En la línea 6 se encuentra el tag <class>, el cual se utiliza para indicarle a JPA las clases a utilizar para la persistencia.
- ✔De las líneas 8 a la 11 se encuentran las configuraciones para conectarse a la base de datos, de igual manera que se realiza con ***Hibernate***. La diferencia es que se cambia el prefijo "***hibernate.connection***" por el de "***javax.persistence***"
- ✔En la línea 12 se encuentra la propiedad para que se creen las tablas.
- ✔En la línea 13 se utiliza la propiedad de ***Hibernate*** para poder ver las sentencias SQL por consola que se van ejecutando en cada transacción al motor de Base de Datos.

HIBERNATE

Usando Hibernate con JPA (EntityManager):

```
1   EntityManagerFactory sessionFactory = Persistence.createEntityManagerFactory("edu.usal.jpa");
2   EntityManager entityManager = sessionFactory.createEntityManager();
3
```

✔Para usar JPA se requiere del objeto ***EntityManager (javax.persistence.EntityManager)*** para poder persistir las entidades.

✔Para la creación del objeto ***EntityManager*** requerimos de la creación del ***EntityManagerFactory (javax.persistence.EntityManagerFactory)*** que a través de un objeto ***Persistence (javax.persistence.Persistence)*** obtendremos la configuración de nuestro ***persistence.xml***

✔Para el mapeo de entidades, seguiremos usando las mismas anotaciones ya vistas. Como estas son propias de ***JPA*** y no del framework desarrollado por ***Hibernate.***

HIBERNATE

Usando Hibernate con JPA (EntityManager):

- ✔ Para persistir una entidad usamos el método ***persist(Object obj)*** que nos provee el ***EntityManager***.
- ✔ Al igual que en Hibernate haremos uso de transacción en el cual en la línea 3 iniciamos la misma y en la línea 5 realizamos el commit o rolback dependiendo de la acción sucedida.
- ✔ Debemos de cerrar el objeto ***EntityManager*** tanto como el ***EntityManagerFactory*** los mismos a través del método ***close()***.

```
3   entityManager.getTransaction().begin();
4   entityManager.persist(new Alumno(2, "Alumno Nombre", "Apellido"));
5   entityManager.getTransaction().commit();
6   entityManager.close();
7   sessionFactory.close();
```

HIBERNATE

Anexo:

- ✔ Documentación oficial: https://hibernate.org/orm/
- ✔ *Descarga v5.4:* *https://sourceforge.net/projects/hibernate/files/hibernate-orm/5.4.32.Final/hibernate-release-5.4.32.Final.zip/download*
- ✔ *Hibernate config:* *https://drive.google.com/file/d/1MPcHobmKNHcNSViNMcV_7Jp_Yg6Ra4Ge/view?usp=sharing*